

National University of Computer and Emerging Sciences



Lab Manual 02 Artificial Intelligence

Course Instructor	Ms. Umm-e-Ammarah
Lab Instructor	Muhammad Hasan
Lab Instructor Email ID	m.hasan@lhr.nu.edu.pk
Semester	Spring 2026

FAST - School of Computing
FAST-NU, Lahore Campus

Contents

Objectives:.....	4
Activities:	4
Activity 1 (Examples):	4
# Using find() to locate the index of the first occurrence of a specified substring within a larger string.	4
# Looking Inside Lists.....	4
# We can even make a list of lists	4
# Lists Like Strings Can Be Sliced Using :	4
# List Modification Examples - List are Mutable.....	5
# List Methods.....	5
# List Comprehensions	5
# Tuples	6
# Tuples Are Like Lists	6
# But List are changeable and Tuples are “immutable”	6
# Things not to do with Tuples	6
# Tuple Functions	6
# Tuples and Assignment	6
# Create Tuple With One Item	7
# Tuples are Comparable	7
# Tuples and Dictionaries - The items() method in dictionaries returns a list of (key,value) tuples	7
# Sorting Lists of Tuples	7
# Sort by Values Instead of Key	7
# The top 2 most common words in input	8
# Python Sets	8
# Python Sets - Duplicate values will be ignored:.....	8
# Set items can be of any data type:	8
# Add an element to a set, using the add() method:	9
# Add elements of a set and list to set:	9
# Remove element by using the remove() method:	9
# Remove element by using the discard() method:	9
# Remove a random item by using the pop() method:	9
# The clear() method empties the set:.....	9
# The del keyword will delete the set completely:	9
Activity 2 (Problem Statements / Tasks):	10

1. Top-N Items Analyzer	10
2. Unique Student Registration System (Sets)	10
3. Number Analysis Using Functions & List Comprehension	10

Objectives:

- ✓ Python Lists
- ✓ Python Tuples
- ✓ Python Dictionary
- ✓ Python Set

Activities:

Activity 1 (Examples):

Note: Python uses zero-based indexing, so the first element has index 0.

Using find() to locate the index of the first occurrence of a specified substring within a larger string.

```
line = input ('Enter text: ')
sub = input ('Enter substring: ')

line = line.strip()
sub = sub.strip()

if line.find(sub) >= 0 :
    print("Substring found at:", line.find(sub))
```

Looking Inside Lists

```
friends = [ 'Ali', 'Ahmad', 'Zain', 'Mustafa', 'Hamza' ]
print(friends[0])    # Ali
print(friends[2])    # Zain
print(friends[-2])   # Mustafa
print(friends[-1])   # Hamza
```

We can even make a list of lists

```
my_list = [
    ['J', 'Q', 'K'],
    ['2', '2', '2'],
    ['6', 'A', 'K'], # (Comma after the last element is optional)
]
print(my_list[0])
print(my_list[2][1])
```

Lists Like Strings Can Be Sliced Using :

```
t = [9, 41, 12, 3, 74, 15]
print(t[1:3])    # All elements from index 1 to 2
print(t[:4])     # All elements from index 0 to 3
print(t[3:])     # All elements from index 3 to last
print(t[:])      # All elements from index 0 to last
print(t)         # All elements from index 0 to last
print(t[1:-1])   # All elements except the first and last
print(t[-3:])    # Last 3 elements
```

List Modification Examples - List are Mutable

```
t = [9, 41, 12, 3, 74, 15]

t[3] = 10
print(t)          # [9, 41, 12, 10, 74, 15]
t[:3] = [1, 2, 3]
print(t)          # [1, 2, 3, 10, 74, 15]
```

List Methods

```
# 'append', 'count', 'extend', 'index', 'insert'
# 'pop', 'remove', 'del', 'reverse', 'sort'
```

List Comprehensions

List Comprehension Example 1

```
squares1 = []
for n in range(10) :
    squares1.append(n**2)

squares2 = [n**2 for n in range(10)]
```

```
print("Square 1 List:", squares1)
print("Square 2 List:", squares2)
```

List Comprehension Example 2

```
planets = ['Mercury', 'Venus', 'Earth', 'Mars', 'Jupiter', 'Saturn']
short_planets1 = []
for planet in planets :
    if len(planet) < 6:
        short_planets1.append(planet.upper() + '!!')

short_planets2 = [planet.upper() + '!!' for planet in planets if len(planet) < 6]

print("Short Planet 1 List:", short_planets1)
print("Short Planet 2 List:", short_planets2)
```

List Comprehension Example 3

```
def count_negatives1(nums):
    negative_nums = []
    for num in nums :
        if num < 0 :
            negative_nums.append(num)
    return len(negative_nums)

def count_negatives2(nums):
    return len([num for num in nums if num < 0])
    # It can also be written as return sum([num < 0 for num in nums])
    # as True + True + True + False + False equals to 3.
```

```
print(count_negatives1([-5, -1, -2, 0, 3]))
print(count_negatives2([-5, -1, -2, 0, 3]))
```

Tuples

```
# often used for functions that have multiple return values.
t = (12, 5, 75)      # it can also be written as: t = 12, 5, 75
print(t)
print(type(t))
```

Tuples Are Like Lists

```
x = ('Ali', 'Ahmad', 'Mustafa')
print(x[2])
print(len(x))
```

```
y = (1, 9, 2)
print(y)
print(max(y))
```

```
for item in y:
    print(item)
```

But List are changeable and Tuples are “immutable”

```
z = (5, 4, 3)
z[2] = 0
```

Things not to do with Tuples

```
x = (3, 2, 1)
x.sort()
```

```
x = (3, 2, 1)
x.append(5)
```

```
x = (3, 2, 1)
x.reverse()
```

Tuple Functions

```
thistuple = (1, 3, 7, 8, 7, 5, 4, 6, 8, 5)
print(thistuple.count(5))  # returns number of times an element exists
```

```
thistuple = (1, 3, 7, 8, 7, 5, 4, 6, 8, 5)
print(thistuple.index(8)) # returns the index of first occurrence of an element
```

Tuples and Assignment

```
(x, y) = (4, 'fred')
print(y)
```

```
(a, b) = (99, 98)
print(a)
```

Create Tuple With One Item

```
thistuple = ("apple",)    # remember the comma
print(type(thistuple))
```

```
thistuple = ("apple")      # not a tuple
print(type(thistuple))
```

Tuples are Comparable

```
print((0, 1, 2) < (5, 1, 2))
print((0, 1, 2000000) < (0, 3, 4))
print(( 'Jones', 'Sally' ) < ( 'Jones', 'Sam' ))
print(( 'Jones', 'Sally' ) > ( 'Adams', 'Sam' ))
```

Tuples and Dictionaries - The items() method in dictionaries returns a list of (key,value) tuples

```
d = dict()
d['a'] = 2
d['b'] = 4
print(d.items())
```

```
for (k,v) in d.items():
    print(k, v)
```

Sorting Lists of Tuples

```
d = {'a':10, 'c':22, 'b':1 }
print(d.items())
print(sorted(d.items()))
```

```
for k, v in sorted(d.items()):
    print(k, v)
```

Sort by Values Instead of Key

```
c = {'a':10, 'b':1, 'c':22, 'd':37}
temp = dict()
```

```
for k,v in c.items():
    temp[v] = k
```

```
print(temp)
print(type(temp))
temp = sorted(temp.items())
print(temp)
print(type(temp))
```

The top 2 most common words in input

```
counts = dict()
line = input("Enter Text: ")
line = line.strip()
words = line.split()

for word in words :
    counts[word] = counts.get(word, 0) + 1

temp1 = dict()
for k, v in counts.items():
    temp1[v] = k
temp1 = sorted(temp1.items(), reverse=True)

for k,v in temp1[:2]:
    print(v, k)
```

For more details, visit [iterate dictionary](#) & [dictionary comprehensions](#)

Python Sets

Set items can appear in a different order every time you use them,
Items cannot be referred to by index or key.
Once a set is created, you cannot change its items
but you can remove items and add new items.

```
myset = {"apple", "banana", "cherry"}
print(myset)
print(len(myset))
print(type(myset))
```

Python Sets - Duplicate values will be ignored:

```
thisset = {"apple", "banana", "cherry", "apple"}
print(thisset)
```

```
thisset2 = {"apple", "banana", "cherry", True, 1, 2}
print(thisset2) # True and 1 is considered the same value
```

Set items can be of any data type:

```
set1 = {"apple", "banana", "cherry"}
set2 = {1, 5, 7, 9, 3}
set3 = {True, False, False}
set4 = {"abc", 34, True, 40, "male"}

print(set1)
print(set2)
print(set3)
print(set4)
```


Add an element to a set, using the add() method:

```
thisset = {"apple", "banana", "cherry"}
thisset.add("orange")
print(thisset)
```

Add elements of a set and list to set:

```
thisset = {"apple", "banana", "cherry"}
tropical = {"pineapple", "mango", "papaya"}
mylist = ["kiwi", "orange"]

thisset.update(tropical)
print(thisset)
thisset.update(mylist)
print(thisset)
```

Remove element by using the remove() method:

```
thisset = {"apple", "banana", "cherry"}
thisset.remove("banana")
# If the item to remove does not exist, remove() will raise an error.
print(thisset)
```

Remove element by using the discard() method:

```
thisset = {"apple", "banana", "cherry"}
thisset.discard("banana")
# If the item to remove does not exist, discard() will NOT raise an error.
print(thisset)
```

Remove a random item by using the pop() method:

```
thisset = {"apple", "banana", "cherry"}
print(thisset.pop())
# when using the pop() method, you do not know which item that gets removed.
print(thisset)
```

The clear() method empties the set:

```
thisset = {"apple", "banana", "cherry"}
thisset.clear()
print(thisset)
```

The del keyword will delete the set completely:

```
thisset = {"apple", "banana", "cherry"}
del thisset
print(thisset)
```

For more details on sets:

<https://docs.python.org/3/library/stdtypes.html#set-types-set-frozenset>

Activity 2 (Problem Statements / Tasks):

1. *Top-N Items Analyzer*

Write a Python program that analyzes item sales and displays the top N most sold items.

Requirements

- a) Use the following dictionary representing item sales:

```
sales = {  
    "apple": 120,  
    "banana": 85,  
    "milk": 200,  
    "bread": 150,  
    "eggs": 90  
}
```
- b) Convert the dictionary into a list of (item, quantity) tuples. Do NOT hardcode results.
- c) Sort the list in descending order of quantity sold using `dict.items()` and `sorted()`
- d) Ask the user to enter a number N.
- e) Display the top N items along with their quantities.

2. *Unique Student Registration System (Sets)*

Create a system that manages unique student registrations for an AI workshop.

Requirements

- a) Repeatedly ask the user to enter student names.
- b) The user types done to stop input.
- c) Store student names in a set to ensure uniqueness.
- d) After input is complete:
 - i. Display:
 - Total number of registered students
 - All registered student names
- e) Ask the user to enter a student name:
 - i. If the name exists, remove it from the set.
 - ii. Otherwise, print "Student not found".

3. *Number Analysis Using Functions & List Comprehension*

Write a Python program that analyzes a list of integers entered by the user.

Requirements

- a) Ask the user to enter integers separated by spaces.
- b) Convert input into a list of integers.
- c) Using functions, compute and return:
 - i. Count of positive numbers
 - ii. Count of negative numbers
 - iii. Count of zeros
- d) Use list comprehension to:
 - i. Create a list of squares of all positive numbers
- e) Display:
 - i. Original list
 - ii. Counts (positive, negative, zero)
 - iii. List of squared positive numbers

Note: At least one function must return multiple values using a tuple.